Mathematical understanding of Linear Regression

It is the first algorithm that any ML developer will ever learn and conceptually it is the easiest one. But mathematically it is a very difficult problem to solve.

First let's talk about what it does...

What linear regression actually do?

I draws a line. That's it.

Let's say you have some data like this.

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
data = pd.DataFrame({
    'feature 1': [1, 2, 3, 4, 5],
    'feature 2': [1.5, 2, 2.5, 3, 3.5],
    'target': [2, 3, 4, 5, 6]
    }
)
data
```

Out[2]:

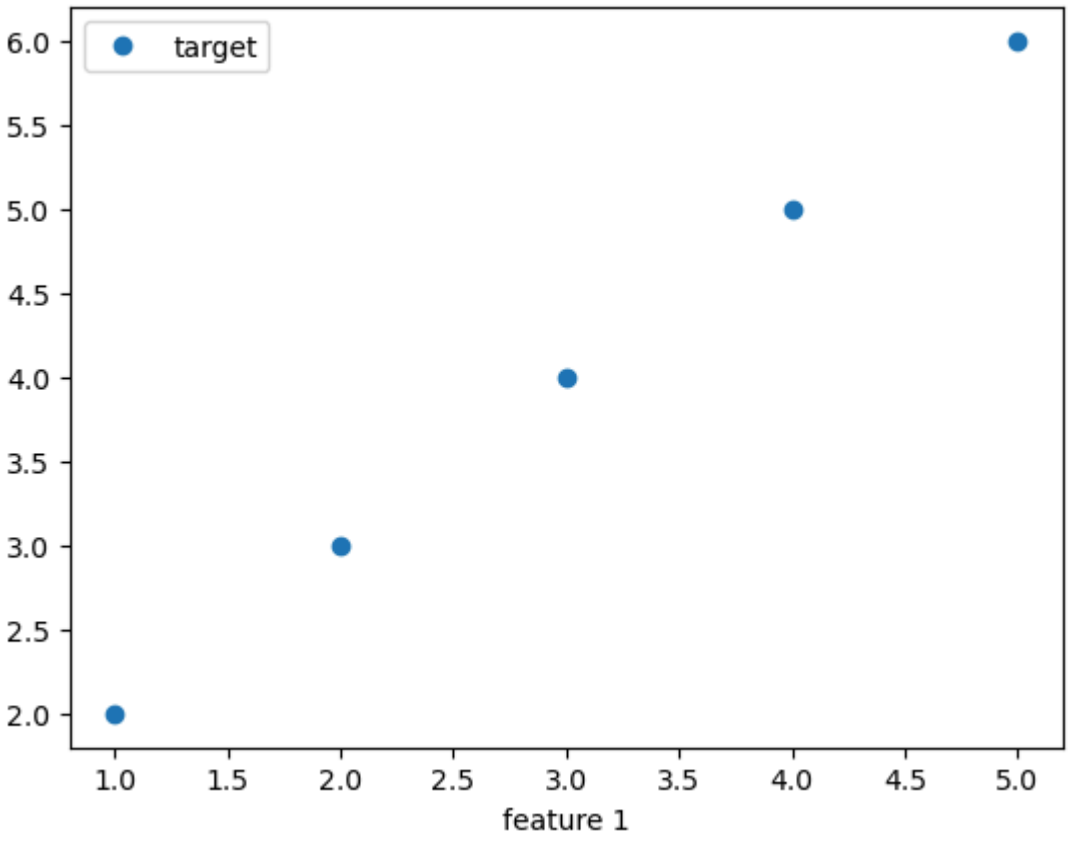
	feature 1	feature 2	target
0	1	1.5	2
1	2	2.0	3
2	3	2.5	4
3	4	3.0	5
4	5	3.5	6

We have two features and one target.

Now, let's try see what `feature 1` vs `target` and `feature 2` vs `target` looks like.

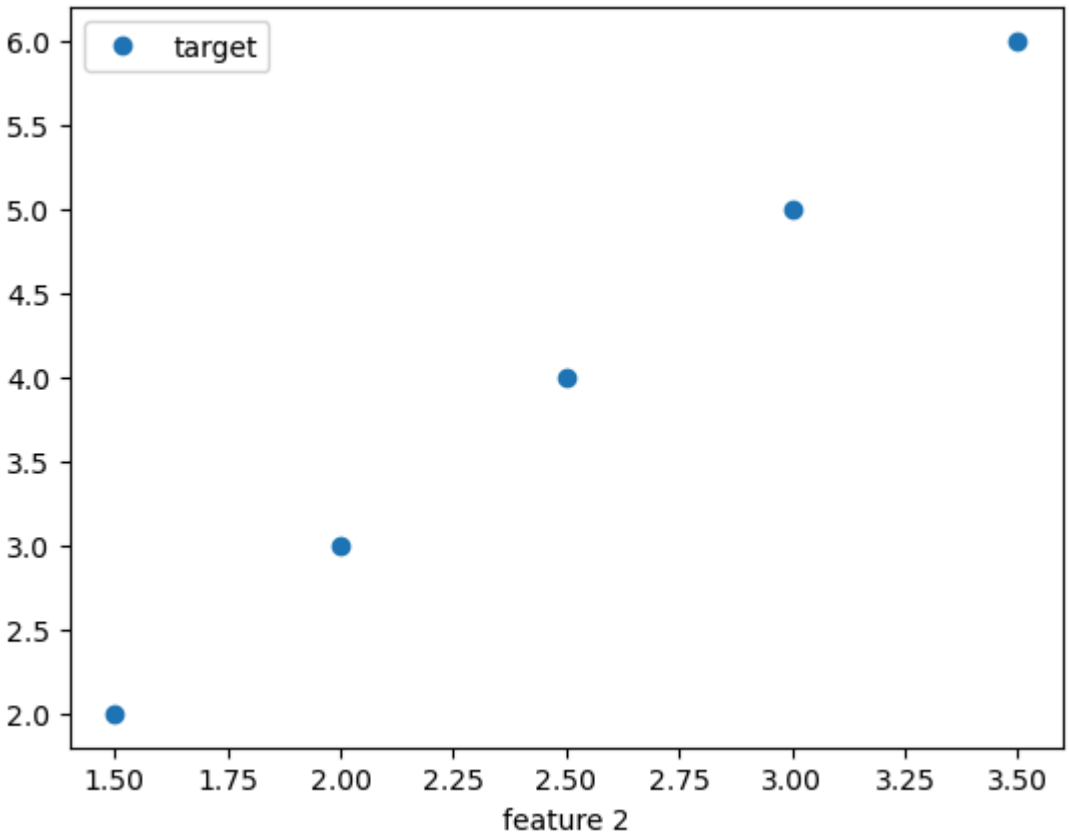
In [3]:
```
data.plot(x='feature 1', y='target', style='o')
```

Out[3]:
```
<Axes: xlabel='feature 1'>
```



In [4]:
```
data.plot(x='feature 2', y='target', style='o')
```

Out[4]:
```
<Axes: xlabel='feature 2'>
```



Both looks identicle but if you look closer `feature 2` has values difference is only `0.5` but `feature 1` has values difference of `1`.

Now you just have to ignore what the values are and ask can I draw a line to represent the relationship between `feature 1` and `target` and `feature 2` and `target`?

The answer is `yes`.

But that's easy to see. But what if I put some noise in the data?

In [5]:
```
np.random.seed(44)
data['feature 1'] += np.random.rand(5)
data['feature 2'] += np.random.rand(5)
data['target'] += np.random.rand(5)
```

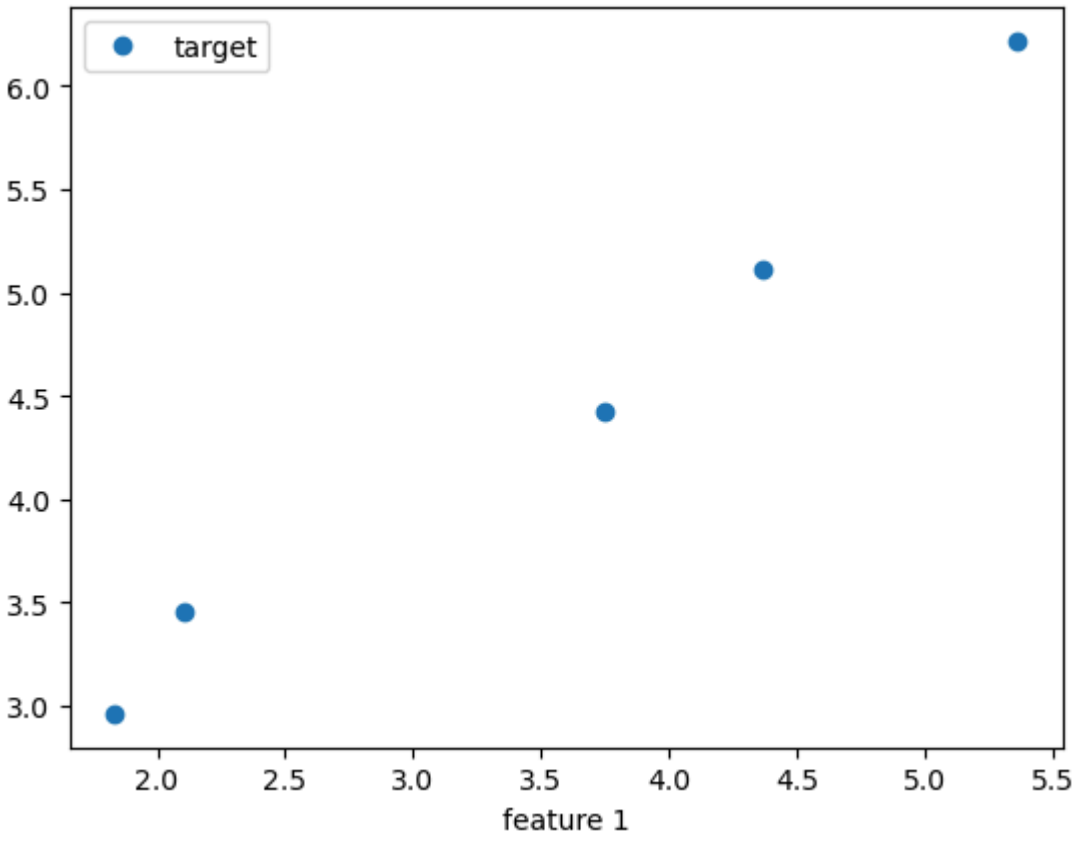
In [6]:
```
data
```

Out[6]:

	feature 1	feature 2	target
0	1.834842	2.109238	2.960526
1	2.104796	2.393780	3.456621
2	3.744640	2.909073	4.427652
3	4.360501	3.509902	5.113464
4	5.359311	4.210148	6.217899

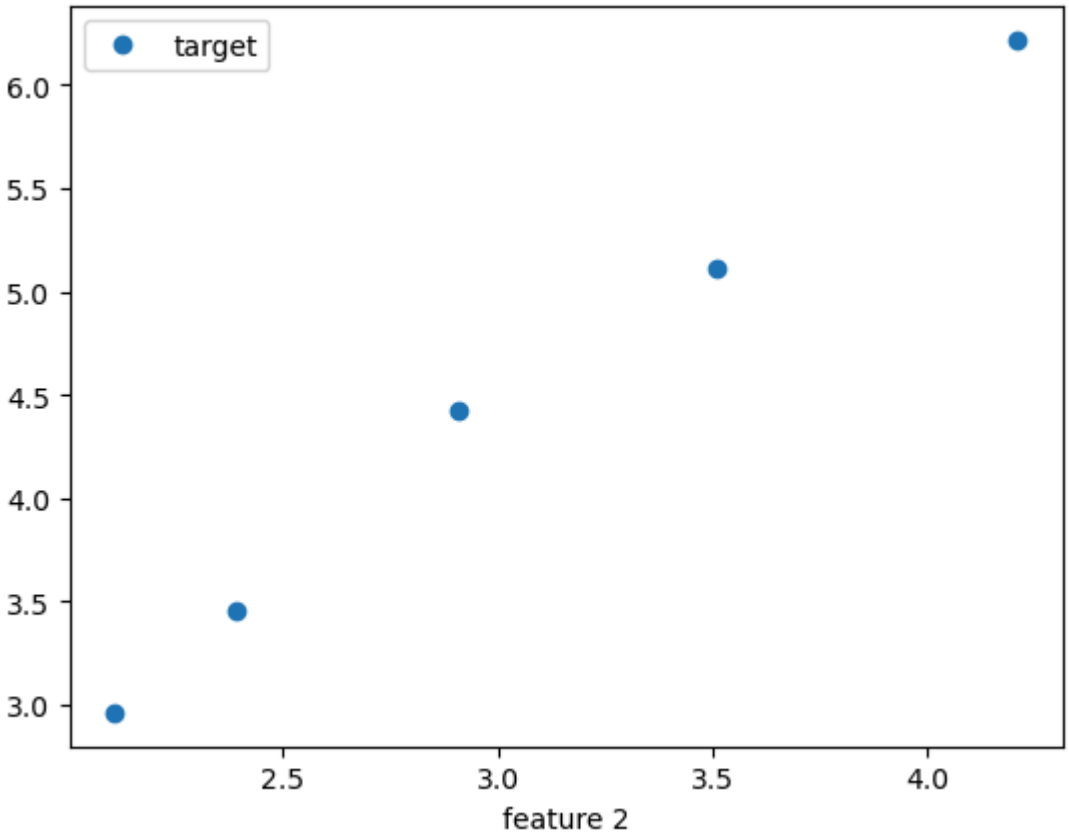
In [7]:
```
data.plot(x='feature 1', y='target', style='o')
```

Out[7]:
```
<Axes: xlabel='feature 1'>
```



In [8]:
```
data.plot(x='feature 2', y='target', style='o')
```

Out[8]: <Axes: xlabel='feature 2'>

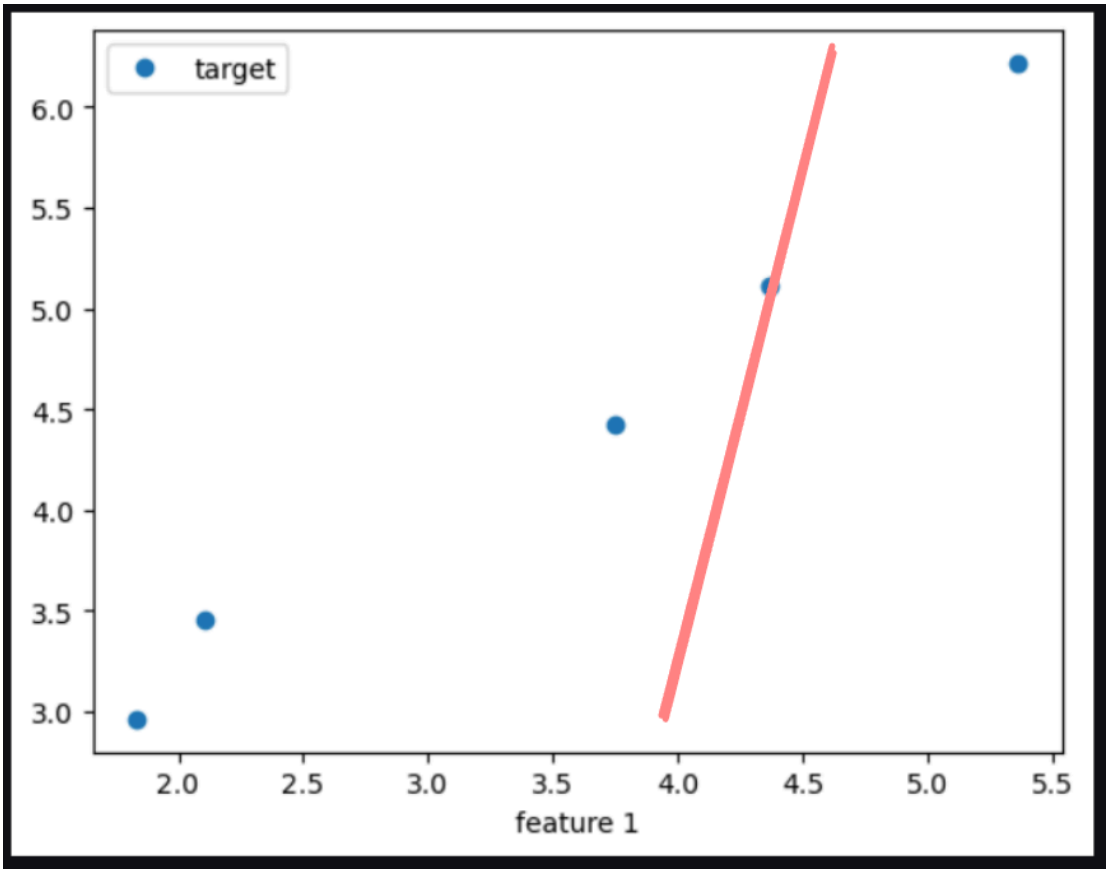


Now, can we draw a line to represent the relationship between `feature 1` and `target` and `feature 2` and `target`?

The answer is still `yes`. The only different is that we have to imagine a line that `best fits` the data.

Now you might ask what does that even mean? `best fits`?

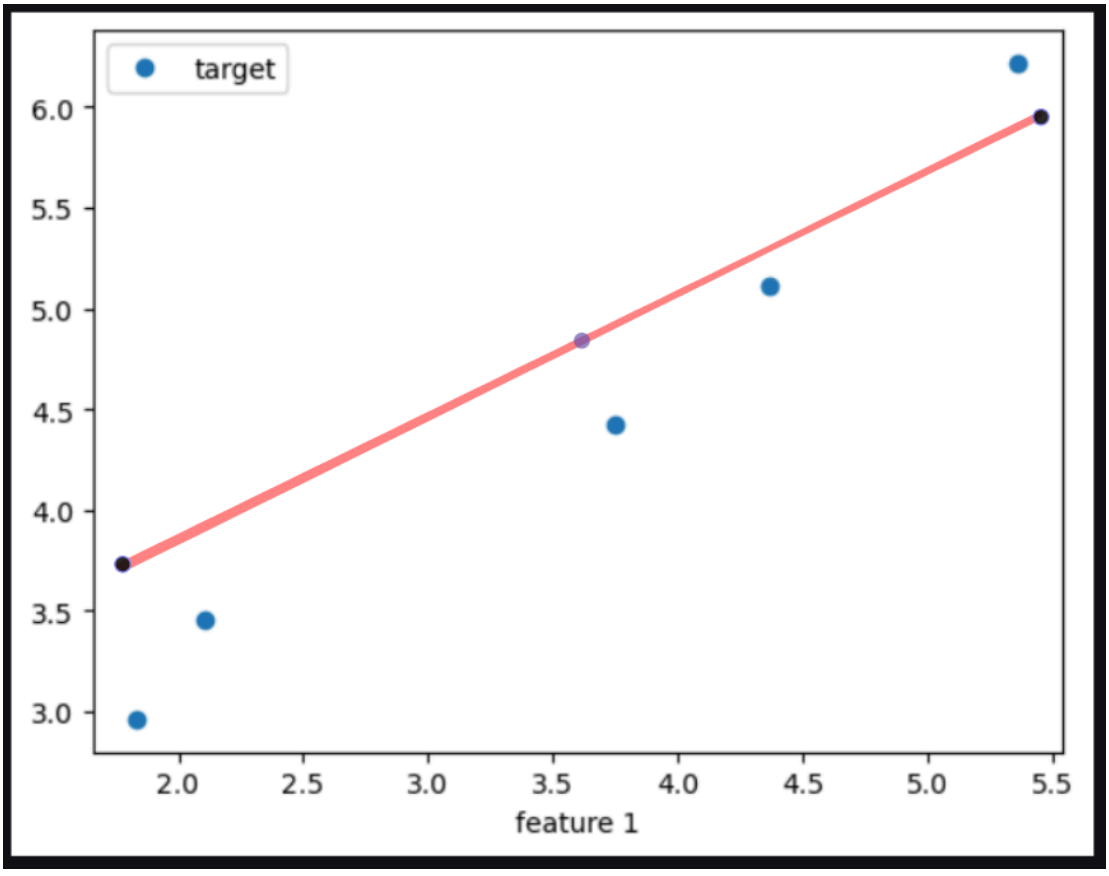
For example, I example let's say I draw line like this.



Does this almost perfectly represent the relationship between `feature 1` and `target`?

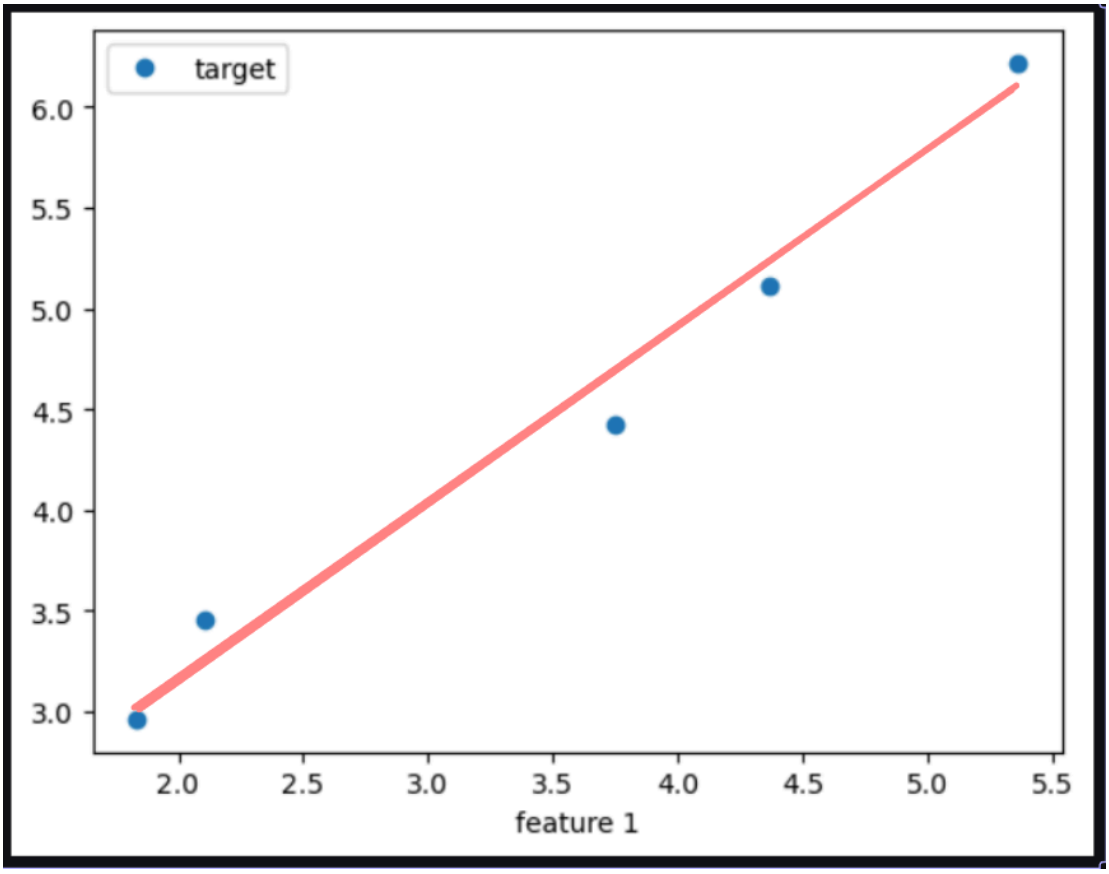
Clearly no.

What if I draw a line like this?



Close close but not quite.

now what about this line.



Looks, pretty good, right?

This is what `best fits` means.

We know that a line cannot represent a non-linear relationship perfectly but we can make it as close as possible.

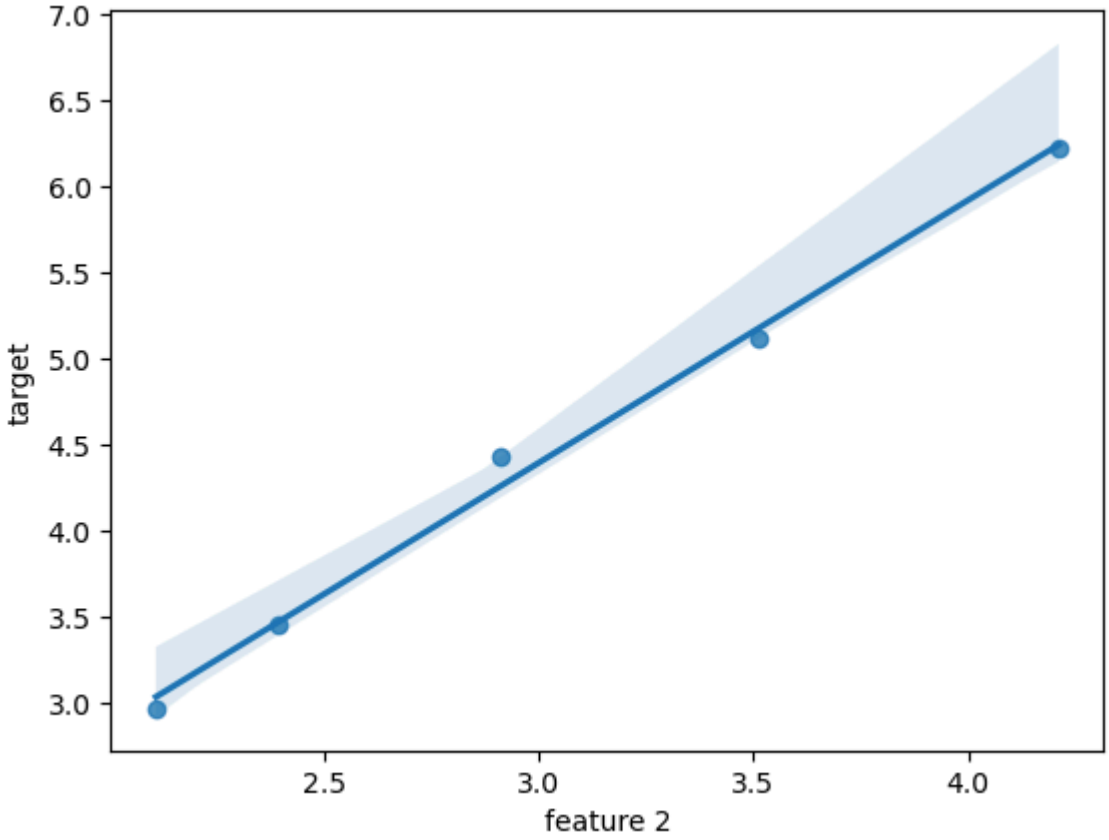
And this is the idea behind the greatest algorithm in all of Machine Learning, `Linear Regression`.

A simple line that will try it's best to represent the relationship between a `feature` and a `target`.

In [9]:
```
import seaborn as sns

sns.regplot(x='feature 2', y='target', data=data)
```

Out[9]:
```
<Axes: xlabel='feature 2', ylabel='target'>
```



The Math

Now, that you have the concept of linear regression, let's go deeper.

Now, we know that we have to find the line that best fits the data.

What is the line ?

But we cannot just tell the computer to manually figure it out.

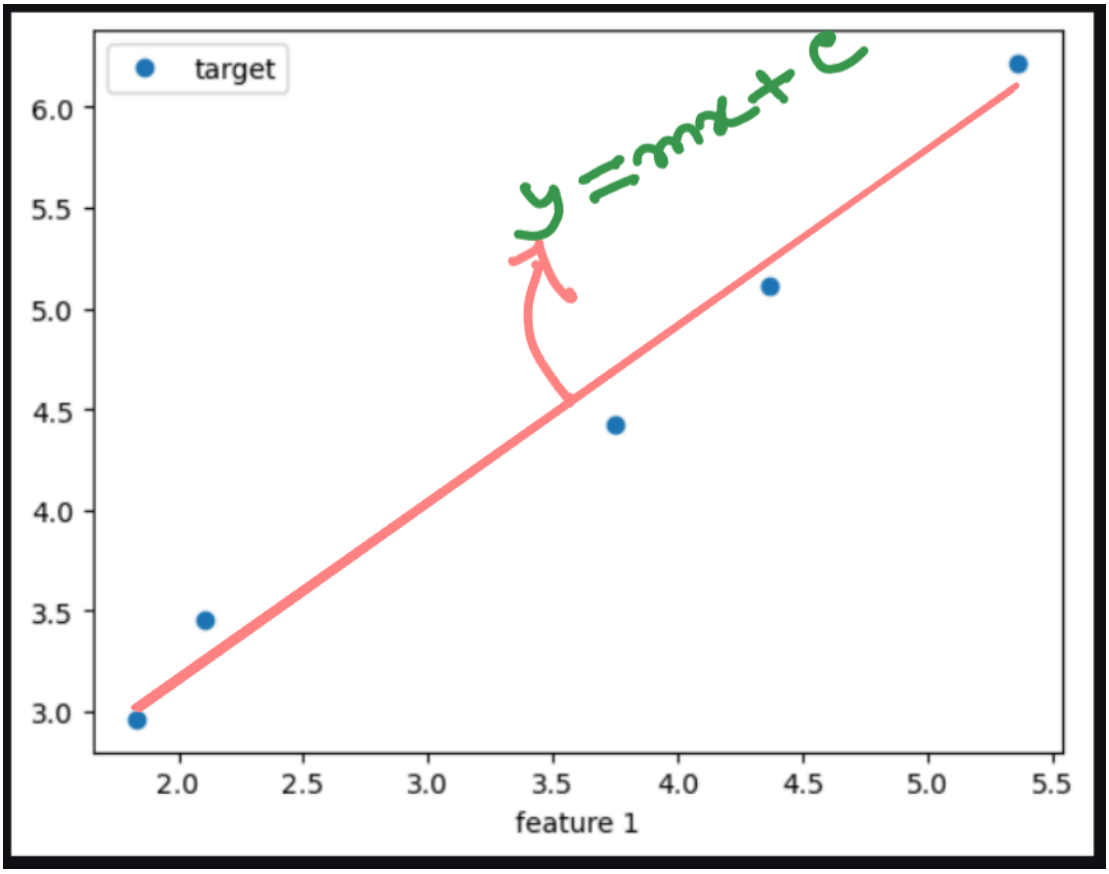
We need a mathematical model.

First let's me ask you a question. What is the mathematical model of a line?

$$y = mx + c$$

right?

$Here,$
$m = slope of the line$
$c = intercept$



Now, as we are talking 1 feature and 1 target, The feature is is the `input` and the target is the `output`.

In mathematical terms, the `input` is represented with `x` and the `output` is represented with `y` and the line is represented with `y = mx + c`

But, by that equation what we are actually predicting is the `approximate output` for the `input` we give it, not the `exact output`.

So, we put a hat on it.

$$\hat{y} = mx + c \qquad (1)$$

Now, to find the line that best fits the data, we need to find the values of m and c and our work is done.

Easy right?

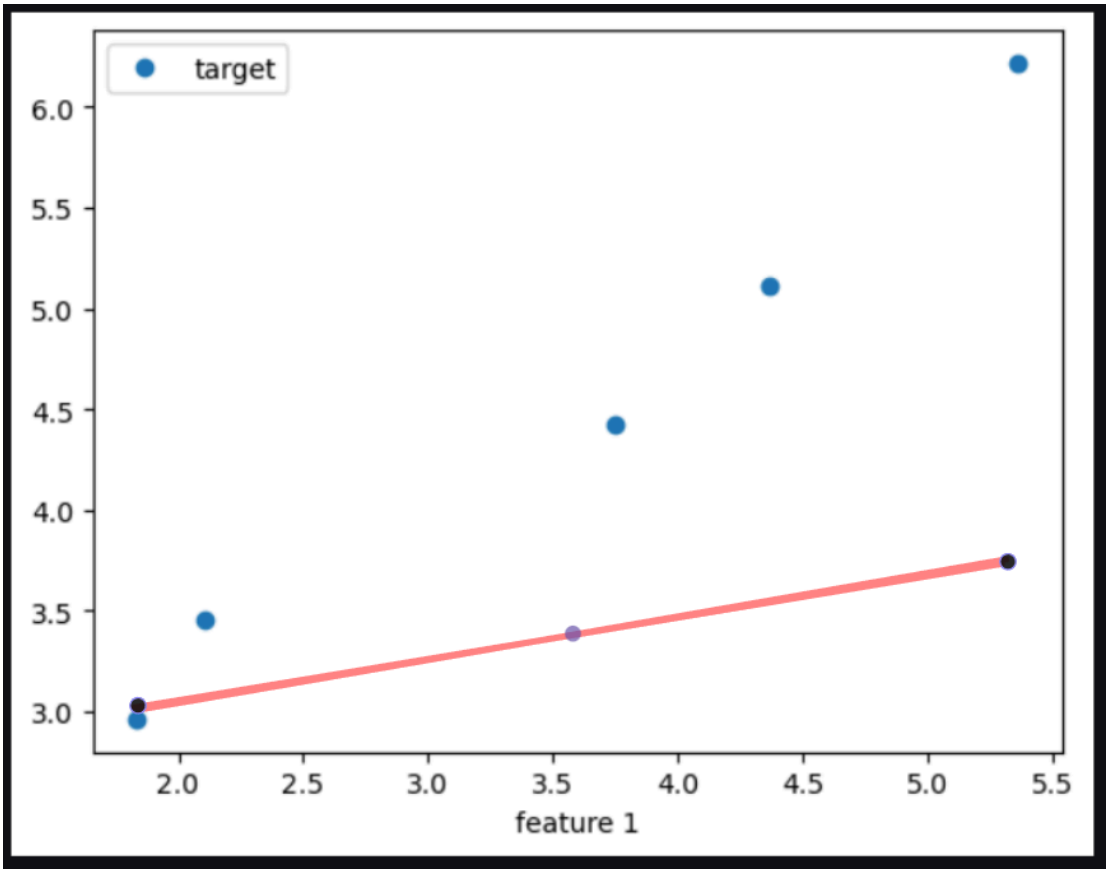
I could've just finish the article here but today I've decided to torture myself to the point of no return.

Where is the line?

Now, how do we figure out the values of m and c?

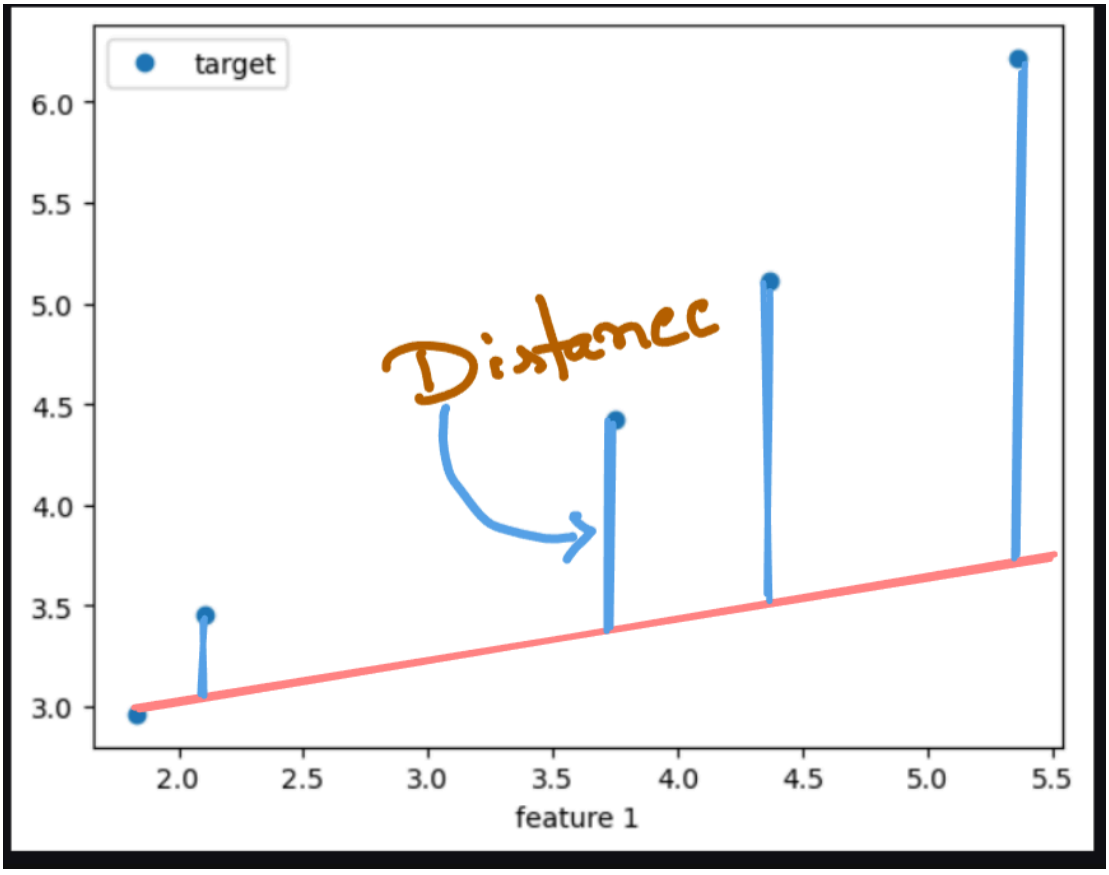
Firs of, all we have to figure out what the fundamental truth of the best fit line is.

Suppose, the line was here.

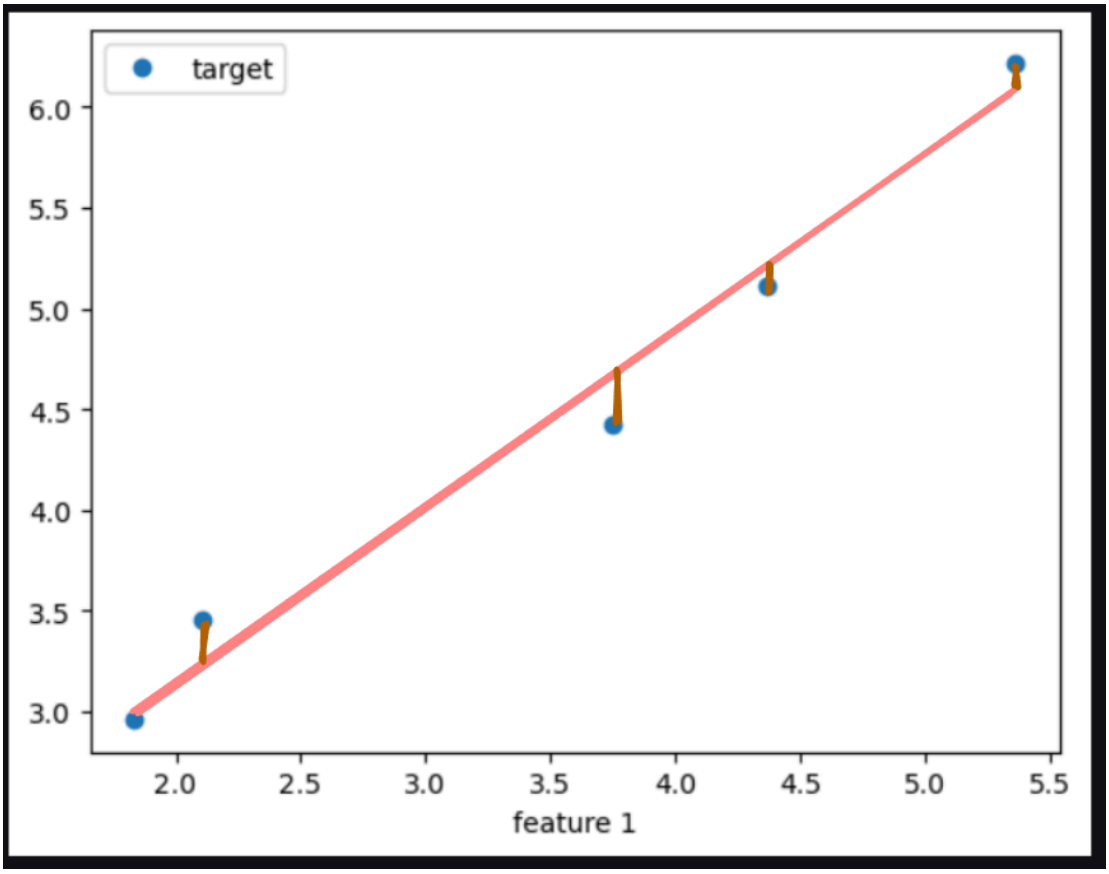


what is the decising factor that tells you whether this is a bad fit?

Distance right? You think how far the line is from the actual values?



A line is better fit if the distance is less.



So, have to figure out the distance of the `approximate output` of the line from the `exact output` of the data.

How do we do that?

$$Distance = y - \hat{y}$$

The weird thing is if you think hard enough this distance also represents the error `approximation` of the line.

So, we can say that the line is the best fit if the the sum of the distance of the `approximate output` $\hat{y}$ of the line, from the `exact output` y of the data is minimum.

In simple terms a line is a best fit when the total error is minimum.

> Error is refered to as `residuals` in statistics.

So, the equation to `total error` is:

$$Error = \sum_{i=1}^{n} (y_i - \hat{y}_i)$$

In actual math a different version of this equation is used.

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \qquad (2)$$

RSS stands for `Residual sum of squares`. Same $error$ equation but squared.

Now you might ask why the squared?

Because for low error values, the total error is low but for high error values, the total error is very high.

This will help the model to get a better optimization.

RSS is the `key` to all fo our answers. We still need to find the values of m and c.

But this is where the mathmatical mumbo jumbo begins.

Finding the values of m and c

We need to find the values of m and c.

We use the (1) and replace the $\hat{y}$ with $mx + c$ in equation (2).

We get,

$$RSS = \sum_{i=1}^{n} (y_i - mx_i - c)^2 \qquad (3)$$

Now, we have a relation between RSS and m and c.

We can think of this equation as $RSS = f(m, c)$.

Now, we think.

What does the minimum of RSS tell us?

This is the total squared error of the best fit line. What is the minimum possible total squared error?

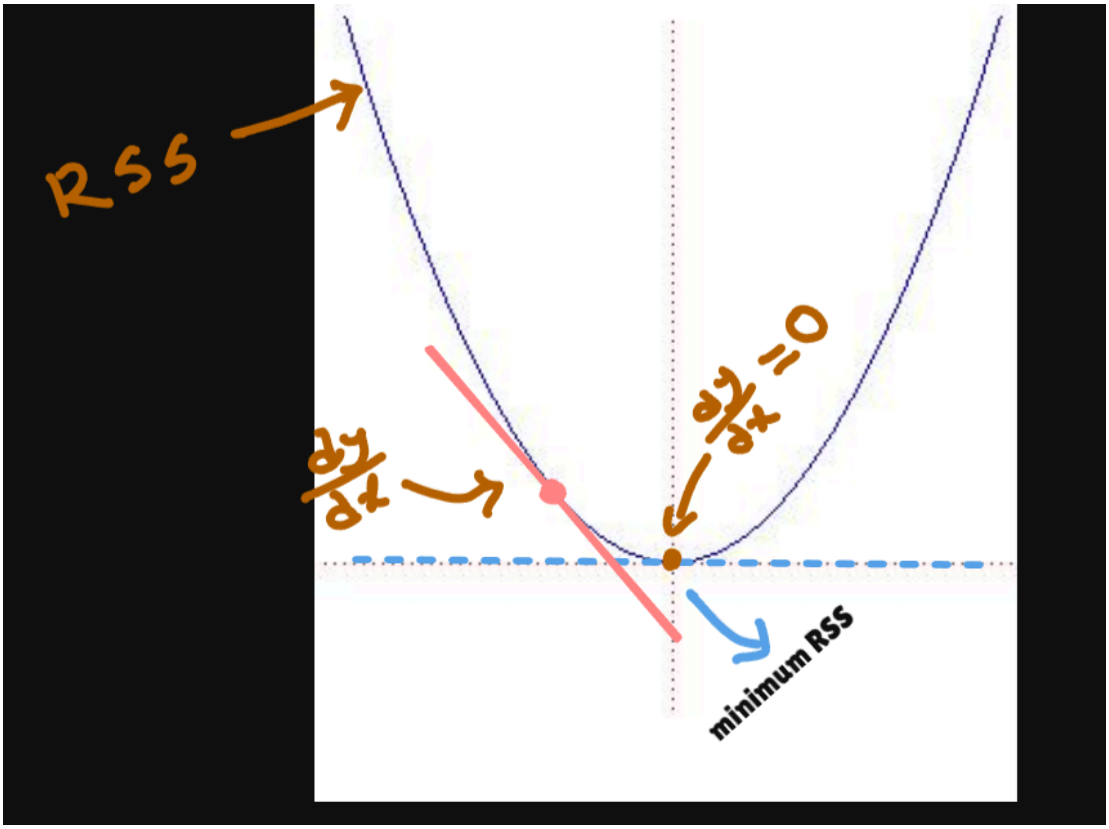
ZERO.

Now, think of gradually decreasing the values of RSS with respect to m or c.

This will form a curve. And The lowest point of the curve is the `minimum` of RSS.

Now, we know that every curve has a slope. So, What should be the slope of the minimum point of the curve?

It's zero.



I think this image should clear things up.

Now, when you are talking about slopes of curves, you are talking about the `derivative` of the curve.

And as we know the slope of the minimum point of RSS is zero.

We can say we will find the minimum point of RSS when the derivative of RSS is zero.

The RSS has two variables m and c. So, we have to do the partial derivative of RSS with respect to m and c.

So, now just do the partial derivative of RSS with respect to m and c and both will be zero at the minimum point of RSS.

$$\frac{\partial RSS}{\partial m} = 0 \qquad (4)$$

$$\frac{\partial RSS}{\partial c} = 0 \qquad (5)$$

Start with

$$RSS = \sum (y_i - mx_i - c)^2$$

Differentiate with respect to c.

$$\frac{\partial RSS}{\partial c} = \sum 2(y_i - mx_i - c)(-1)$$

which becomes

$$-2\sum(y_i - mx_i - c)$$

Set equal to zero.

$$-2\sum(y_i - mx_i - c) = 0$$

Divide by -2.

$$\sum(y_i - mx_i - c) = 0$$

If we expand.

$$\sum y_i - m\sum x_i - \sum c = 0$$

Since there are (n) copies of (c),

$$\sum c = nc$$

Therefore

$$\sum y_i - m\sum x_i - nc = 0$$

Solve for (c).

$$c = \frac{\sum y_i - m\sum x_i}{n}$$

Notice

$$\bar{x} = \frac{\sum x_i}{n}$$

and

$$\bar{y} = \frac{\sum y_i}{n}$$

Therefore

$$\boxed{c = \bar{y} - m\bar{x}}$$

This is our first important equation.

Now, `Differentiate with respect to m`

Again,

$$RSS = \sum(y_i - mx_i - c)^2$$

Differentiate.

Using the chain rule,

$$\frac{\partial RSS}{\partial m} = \sum 2(y_i - mx_i - c)(-x_i)$$

Simplify.

$$-2\sum x_i(y_i - mx_i - c)$$

Set equal to zero.

$$\sum x_i(y_i - mx_i - c) = 0$$

Expand.

$$\sum x_i y_i - m\sum x_i^2 - c\sum x_i = 0$$

Move terms.

$$m\sum x_i^2 = \sum x_i y_i - c\sum x_i$$

Now substitute

$$c = \bar{y} - m\bar{x}$$

Substitute c

Substitute into the equation.

$$m\sum x_i^2 = \sum x_i y_i - (\bar{y} - m\bar{x})\sum x_i$$

Expand.

$$m\sum x_i^2 = \sum x_i y_i - \bar{y}\sum x_i + m\bar{x}\sum x_i$$

Move every (m) term to one side.

$$m\left(\sum x_i^2 - \bar{x}\sum x_i\right) = \sum x_i y_i - \bar{y}\sum x_i$$

Since

$$\sum x_i = n\bar{x}$$

we obtain

$$m\left(\sum x_i^2 - n\bar{x}^2\right) = \sum x_i y_i - n\bar{x}\bar{y}$$

Finally,

$$\boxed{m = \frac{\sum x_i y_i - n\bar{x}\bar{y}}{\sum x_i^2 - n\bar{x}^2}}$$

A Cleaner Formula

Using the identities

$$\sum(x_i - \bar{x})(y_i - \bar{y}) = \sum x_i y_i - n\bar{x}\bar{y}$$

and

$$\sum(x_i - \bar{x})^2 = \sum x_i^2 - n\bar{x}^2$$

the slope becomes

$$\boxed{m = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}}$$

This is the version you'll most often see in statistics and machine learning texts.

Compute the Intercept

Now substitute (m) back into

$$\boxed{c = \bar{y} - m\bar{x}}$$

and you're done.

The Final Equations

The optimal slope is

$$\boxed{m = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}}$$

The optimal intercept is

$$\boxed{c = \bar{y} - m\bar{x}}$$

The regression line is

$$\boxed{\hat{y} = mx + c}$$

And that's it.

We have m and c.

And buuuut!

This is only for `one feature`.

If you have more than one feature, you will have to calculate the slope and intercept for each feature.

That'll be a bigger problem. But I've forced myself to get into all the mathematics. So, here I go again.

Linear regression with multiple features

When we have a single feature and a single target we have to find the regression line $y = mx + c$.

But nost of the time we will not use a single feature and a single target. There will be multiple features and all those feature will have a correlation with the target, that means a regresison line should be drawn for each feature.

Each regression line will have a slope and an intercept. So, we have to calculated for each feature and each target.

So, the regression like equation will become:

$$\hat{y} = m1x_1 + m2x_2 + m3x_3 + \ldots + c$$

> Here, c represents the summation of the intercepts because intercepts are just constants.

The notations used here is for convienience. But if you've come this far into the article I think using the proper ML lingo won't hurt.

I'll use b_0 for intercept and b_i where $i = 1, 2, 3, \ldots$ for slope.

So for p features the equation will become:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \ldots + b_px_p$$

If the number of features rises the calculation will become more complicated.

Now, to figure out what we can we do to stay away from all the complecations?

Time for a though experiment.

Suppose,

We have a data set like this.

Size	Bedrooms	Age	Price
1200	2	15	250
1500	3	10	320
1800	4	5	400

the dataset has 3 features and 1 target.

So, quation for the regression line should be,

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3$$

b_1 represents the slope of the first feature. b_2 represents the slope of the second feature. b_3 represents the slope of the third feature.

So, if we think of the dataset as testing data, then each row will be passed like this,

$$\hat{y}_1 = b_0 + 1200b_1 + 2b_2 + 15b_3$$
$$\hat{y}_2 = b_0 + 1500b_1 + 3b_2 + 10b_3$$
$$\hat{y} + 3 = b_0 + 1800b_1 + 4b_2 + 5b_3$$

Now, if you know about matrixes, then instead of separtely writting out all the equations, you can write it in matrix form.

$$\begin{bmatrix} 1 & 1200 & 2 & 15 \\ 1 & 1500 & 3 & 10 \\ 1 & 1800 & 4 & 5 \end{bmatrix} \times \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \end{bmatrix}$$

Now, you might ask how is this even possible?

> check out this matrix multiplication video.

If we multiply the matrix on the left with the matrix on the right we will get.

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \end{bmatrix} = \begin{bmatrix} b_0 + 1200b_1 + 2b_2 + 15b_3 \\ b_0 + 1500b_1 + 3b_2 + 10b_3 \\ b_0 + 1800b_1 + 4b_2 + 5b_3 \end{bmatrix}$$

It's the same equations we had before. So, instead of writing out all the equations, we can write it in matrix form.

Now, we do what we did before to figure out the coefficients but with matrix now. NOOOOOOOOO.

Matrix notations

$$\begin{bmatrix} 1 & 1200 & 2 & 15 \\ 1 & 1500 & 3 & 10 \\ 1 & 1800 & 4 & 5 \end{bmatrix} \times \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \end{bmatrix}$$

Here the first matrix is the whole datset with a extra column of 1's.

We can say,

$$X = \begin{bmatrix} 1 & 1200 & 2 & 15 \\ 1 & 1500 & 3 & 10 \\ 1 & 1800 & 4 & 5 \end{bmatrix}$$

The second matrix is the coefficients.

$$\beta = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

We can rename the predicted value matrix as $\hat{y}$.

So, the final equation becomes,

$$\hat{y} = X\beta$$

Now, we need the residuals or the error.

Previously,

$$e_i = y_i - \hat{y}_i$$

Now we have matrix. And in matrix the substraction is done in one to one correspondence. So, we can find the residuals matrix as,

$$\begin{bmatrix} e_0 \\ e_1 \\ e_2 \\ \vdots \\ e_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} - \begin{bmatrix} \hat{y}_0 \\ \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$

$$e = y - \hat{y}$$

Here, y is the exact output matrix and $\hat{y}$ is the approximate output matrix.

Since, we saw $\hat{y} = X\beta$, we can say,

$$e = y - X\beta$$

Now, the pain begins again.

We need to find the `RSS`

RSS

Previously for only one feature,

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

But how we we re-write this with matrix?

We cannot just sqaure the `residuals` matrix. Matrix mutliplication works diffrently...

What is a square? It's a value that is multiplied by itself right?

So, in matrix we cannot just derectly multiply the the same matrix with itself. We have to transpose the matrix first and then multiply.

So, we have to do,

$$e = y - X\beta$$

Square it would be,

$$e^T e = \begin{bmatrix} e_1, e_2, \cdots, e_n \end{bmatrix} \begin{bmatrix} e_0 \\ e_1 \\ e_2 \\ \vdots \\ e_n \end{bmatrix}$$

Now if we do the multiplication.

We will get.

$$e_1^2 + e_2^2 + \cdots + e_n^2$$

or simply,

$$\sum_{i=1}^{n} e_i^2$$

or

$$\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

So, the simple $e^T e$ is the sum of the square of the residuals.

So, we can directly write the `RSS` as,

$$RSS = e^T e$$

Here, T is not an exponent symbol. It is a transpose symbol.

Now, we can replace the e with $y - X\beta$ and we get.

$$RSS = (y - X\beta)^T (y - X\beta)$$

And this is the matrix version of the `RSS` equation.

Now, we need to solve for β. Because that is our goal finding all the coefficients for the features.

Brace yourself. Thinks are gonna get a whole lot unga-bunga.

Simplifying the equation

First of all do not forget β is a vector where all the coefficients are stored.

The RSS equation is still too complex to differentiate.

So, we need to simply it more.

In matrix,

$$(A - B)^T = A^T - B^T$$

and

And In matrices,

$$(AB)^T = B^T A^T$$

So, we can rewrite,

$$(y - X\beta)^T = y^T - \beta^T X^T$$

Therefore,

$$RSS = (y^T - \beta^T X^T)(y - X\beta)$$

Now, we can separate like normal algebra.

$$RSS = y^T - y^T X\beta - \beta^T X^T y + \beta^T X^T X\beta$$

Now, here's the twist. The middle two terms are actually scalers.

> Scaler is a 0-dimentional Matrix. Simply, it's a matrix with only one element.

How, you ask?

In the middle we have $y^T X\beta$ and $\beta^T X^T y$

If you take,

$$y = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

It's a `n x 1` matrix. n is the number of `samples`.

We are multiplying a y^T with X and then with beta.

So, the y^T will be a `1 x n` matrix. X will be a `n x p` matrix.

If we multiply, we will get a `1 x p` matrix.

And beta will be a `p x 1` matrix.

after all the multiplications, we will get a `1 x 1` matrix.

This means it is a scaler.

if you look closely. $\beta^T X^T y$ is nothing but $y^T X \beta$ transposed.

Now, here's mor einfo dump. A scaler is always equals to it's transpose.

So, we can write.

$$y^T X \beta = \beta^T X^T y$$

So, the RSS equation becomes,

$$RSS = y^T y - 2y^T X \beta + \beta^T X^T X \beta$$

And, now we have a more simplified RSS equation that we can differentiate.

Deriving the β

You need to know two matrix identities:

$$\frac{\partial}{\partial \beta}(a^T \beta) = a$$

and

$$\frac{\partial}{\partial \beta}(\beta^T A \beta) = 2A\beta$$

This only applies if `A` is symmetric.

> You can study this for more detailed information.

Now time for some calculus.

partial differentiating the RSS equation with respect to β,

$$\frac{\partial}{\partial \beta} RSS = \frac{\partial}{\partial \beta} y^T y - \frac{\partial}{\partial \beta} 2y^T X \beta + \frac{\partial}{\partial \beta} \beta^T X^T X \beta$$

$$\frac{\partial}{\partial \beta} RSS = 0 - 2\frac{\partial}{\partial \beta}(X^T y)^T \beta + \frac{\partial}{\partial \beta}(\beta^T X^T X \beta)$$

Now here's a fun fact for you, $A^T A$ is always symmetric. And now, we can directly use those 2 identities.

$$\frac{\partial}{\partial \beta} RSS = -2x^T y + 2X^T X \beta$$

Now, if the the RSS is minimum the derivative of the RSS is zero. So, we can just set the derivative to zero.

$$-2X^Ty + 2X^TX\beta = 0$$

or we can just say,

$$X^TX\beta = X^Ty$$

Now we have to

Solve for β

We multiply the above equation with $(X^TX)^{-1}$ this will eliminate the X^TX term from the left side giving us only β.

and we get,

$$\beta = (X^TX)^{-1}X^Ty$$

And this is the Normal Equation.

> If you dont know how inverse matrix works check out this site.

And we are done.

Validating the Normal Equation

You might think this is a different quation from before but this is exactly the same thing btu in matrix form.

For example, if you have one feature,

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

and

$$\beta = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

If you compute

$$(X^TX)^{-1}X^Ty,$$

you recover exactly the same formulas you derived earlier:

$$b_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2},$$

and

$$b_0 = \bar{y} - b_1\bar{x}.$$

So the Normal Equation is not a new method, it's the **generalization of the least-squares derivation to any number of features**.

OMG! That was a lot of math.

A LOT OF MATH!

But why not torture my self more but making the whole thing in numpy and test it

out?!

Implementing Linear Regression

You need python installed in your computer and then open your terminal and write,

```
pip install numpy
```

What is Numpy

It's python library optimed heavily for numerical and multi-dimensional array operations.

Normal python would not be able to handle the amount of data a machine learning model needs to handle.

So, here's the full code.

In [10]:
```
import numpy as np
```

```
class LinearRegression:

    def __init__(self):
        # the beta values
        self.weights = None

    def fit(self, X, y):

        # preparing the X matrix
        ones = np.ones((X.shape[0],1))
        X = np.hstack((ones,X))

        # we will need X transpose
        XT = X.T

        # we will need X transpose multiplied by X
        XTX = XT @ X

        # we need the inverse of the X transpose multiplied by X
        inverse = np.linalg.inv(XTX)

        # this line will calculate X transpose multiplied by y
        XTy = XT @ y

        # this is the normal equation we found
        self.weights = inverse @ XTy

    def predict(self, X):

        # The prediction function follow the normal equation
        ones = np.ones((X.shape[0],1))
        X = np.hstack((ones,X))

        # the predicted value will be the dot product of X and the weight
        return X @ self.weights
```

And that's it.

We have a nice little prototype of a linear regression model.

Time to test it. First let's test it with a single feature.

In [11]:
```
X = np.array([
    [1],
    [2],
    [3],
    [4],
    [5]
])

y = np.array([3,5,7,9,11])
```

The above code has a relationship of

$$y = 2x + 1$$

Here $b_0 = 1$ and $b_1 = 2$. It is a linear relationship. So, let's see if the model is able to calculate the same values.

In [12]:
```
model = LinearRegression()

model.fit(X,y)
```

In [13]:
```
model.weights
```

Out[13]:
```
array([1., 2.])
```

YES!! it did.

Let's try to predict something.

In [14]:
```
model.predict(np.array([[6]]))
```

Out[14]:
```
array([13.])
```

IT WOOOORRRKS!

This is exactly what we get for $y = 2x + 1$. Here, x is 6, so y = 13.

Now, Let's try for multiple features.

In [15]:
```
import numpy as np

X = np.array([
    [10, 2, 20],
    [12, 3, 18],
    [15, 3, 15],
    [18, 4, 12],
    [20, 4, 10],
    [22, 5, 8],
    [25, 5, 6],
    [28, 6, 5],
])

y = np.array([
    30,
    50,
    65,
    90,
    100,
    120,
    132,
    151
])
```

The above data represents,

$$y = 50 + 2x_1 + 10x_2 - 3x_3$$

The coefficients are $b_0 = 50, b_1 = 2, b_2 = 10, b_3 = -3$

In [16]:
```
model.fit(X,y)

model.weights
```

Out[16]: array([50., 2., 10., -3.])

Exactly what we wanted.

In [17]:
```
sample = np.array([[30, 6, 4]])

prediction = model.predict(sample)

print(prediction)
```

```
[158.]
```

Wow, we did it...

Everything works just fine.

I would'be been so happy to end this here. But! remember there's always a BUTT!

Complexity

Even though everything seems just fine as a programmer, efficiency is the most important thing.

complexity actually depends on:

- (n) = number of training samples
- (p) = number of features

Since in Machine Learning, usually

$$n \gg p$$

we should analyze using both variables.

The fit() method

```
def fit(self, X, y):

    ones = np.ones((X.shape[0],1))
    X = np.hstack((ones,X))

    XT = X.T

    XTX = XT @ X

    inverse = np.linalg.inv(XTX)

    XTy = XT @ y

    self.weights = inverse @ XTy
```

Assume

- (n) = number of samples

- (p) = number of features

After adding the bias column,

$$X$$

has dimensions

$$n \times (p+1)$$

For simplicity, we'll just write

$$n \times p$$

since adding one column doesn't affect asymptotic complexity.

Step 1

```
ones = np.ones((X.shape[0],1))
```
Creates n ones.

Time

$$O(n)$$

Memory

$$O(n)$$

Step 2

```
X = np.hstack((ones,X))
```
NumPy cannot simply "attach" a column.

It allocates an entirely new matrix.

New size

$$n \times (p+1)$$

Every element must be copied.

Time

$$O(np)$$

Memory

$$O(np)$$

Step 3

```
XT = X.T
```

Here's something many people don't know.NumPy transpose is almost free. It doesn't copy the matrix. It simply changes the **strides**.

Time

$$O(1)$$

Memory

$$O(1)$$

Unless a later operation explicitly requires a contiguous copy.

Step 4

```
XTX = XT @ X
```
Dimensions

$$(p \times n)(n \times p)$$

Result

$$p \times p$$

For every entry,

we have to compute n multiplications.

There are p^2 entries.

Therefore

$$\boxed{O(np^2)}$$

Memory Stores a $p \times p$ matrix.

$$O(p^2)$$

Step 5

```
inverse = np.linalg.inv(XTX)
```
Now we're inverting $p \times p$ matrix. Modern libraries use optimized LU decomposition.

Complexity

$$\boxed{O(p^3)}$$

Memory

$$O(p^2)$$

Step 6

`XTy = XT @ y`
Dimensions

$$(p \times n)(n \times 1)$$

Result $p \times 1$.

Each of the p rows performs n operations.

Complexity

$$\boxed{O(np)}$$

Memory

$$O(p)$$

Step 7

`self.weights = inverse @ XTy`
Dimensions

$$(p \times p)(p \times 1)$$

Complexity

$$\boxed{O(p^2)}$$

Memory

$$O(p)$$

Putting Everything Together

Operation	Time Complexity	Memory
Create Ones	$O(n)$	$O(n)$
Add Bias Column	$O(np)$	$O(np)$
Transpose	$O(1)$	$O(1)$
(X^TX)	$O(np^2)$	$O(p^2)$
Matrix Inverse	$O(p^3)$	$O(p^2)$
(X^Ty)	$O(np)$	$O(p)$
Final Multiplication	$O(p^2)$	$O(p)$

Overall Time Complexity

Adding everything,

$$O(n) + O(np) + O(np^2) + O(p^3) + O(np) + O(p^2)$$

Removing lower-order terms,

$$\boxed{O(np^2 + p^3)}$$

> Well, that's not good.

This is the true complexity of solving linear regression using the Normal Equation.

Space Complexity

The largest object stored is the feature matrix itself,

X

which requires $O(np)$ memory.

Other matrices like $X^T X$ only require $O(p^2)$ memory.

which is typically much smaller when (n \gg p).

Thus the overall auxiliary space is

$\boxed{O(np + p^2)}$,

or simply

$\boxed{O(np)}$

when (n) is much larger than (p).

Although the Normal Equation is written as

$$\beta = (X^T X)^{-1} X^T y,$$

high-quality libraries like scikit-learn typically **do not** compute the inverse explicitly. Computing a matrix inverse is slower and can be numerically unstable when features are highly correlated.

Instead, they solve the equivalent linear system

$$(X^T X)\beta = X^T y$$

using matrix factorizations such as **QR decomposition**, **SVD**, or direct linear solvers. These approaches produce the same solution while being more numerically stable and often more efficient in practice.

> That's why the Normal Equation is best thought of as a mathematical derivation rather than the exact algorithm used in production libraries.

Final Verdict

I always wanted to make a article on the deep mathematical underpinnings of Machine Learning algorithm. Even though i showed the mathematical derivation of Linear Regression, it is not the best way of finding regression coefficients.

I don't want to go deeper into that. I think that thing needs it's own book. That's what puts the `learning` into `machine learning`.

Will talk about how a machine can actually learn in the next article.

Thank you for reading.